

Tourism Government — Ultimate Technical Architecture & Module Mastery Manual

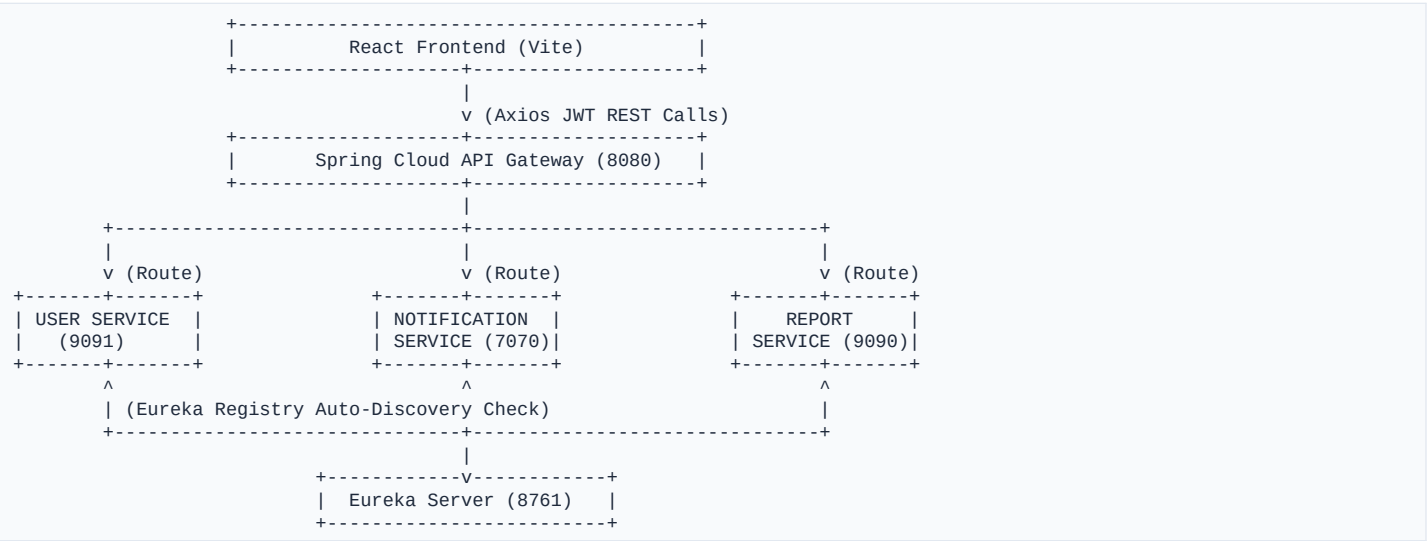
10+ Pages of Detailed Specifications, Code Walks, Annotations, Database Schemas, UML Flows, & Verbal Presentation Script

■ **ULTIMATE microserviceS BLUEPRINT:**

This handbook is the ultimate technical asset for your final project presentation and architect-level interviews. It focuses exclusively on your modules (Dashboard, Notification, and Reports). It features absolute details on the technology stack, Spring Boot annotations dictionary, method-level code walks, UML sequence representations, sliding-window fault tolerance, database designs, and a complete 5-Minute Spoken Presenter Script.

Chapter 1: Complete Project Architecture & Module Overview

Our Tourism Government System uses a state-of-the-art **Microservices Architecture** to guarantee absolute scalability and system uptime. The system separates core concerns into decoupled microservices, communicating dynamically through a central gateway and registered on a central lookup server.



Key Pillars of the Architecture:

- **Database-per-Service:** Each microservice maintains its own distinct database schema (e.g. `notificationdb`, `reportdb`). No database tables are shared, ensuring that a database schema change in one service will never break other services.
- **API Gateway Routing:** All frontend Axios requests hit the Gateway port `8080` which validates the JWT tokens and routes them.
- **Dynamic Lookup:** Eureka acts as our central lookup registry, matching microservice names to active IP/port locations dynamically.

Chapter 2: Absolute Tech Stack Directory

Every single technology used in your modules is chosen for maximum performance, clean design, and structural resilience:

Technology / Library	Where Used & Architectural Role
Spring Boot (v3.x)	The core backend framework. Powers our controllers, services, repositories, and dependency injections.
Spring Cloud OpenFeign	Declarative HTTP client. Used by `ReportService` to call `SiteClient` and `UserClient` dynamically.
Resilience4j	Fault tolerance engine. Implements sliding window circuit breakers and fallback classes directly on Feign interfaces.
Spring Data JPA & Hibernate	ORM layer that maps our notification and report Java entities directly into MySQL database tables seamlessly.
MySQL Connector	Driver used by microservices to establish connections to local schemas (`notificationdb`, `reportdb`).
Lombok	Reduces boilerplate by automatically generating getters, setters, constructors, and builders during compile time.
React (Vite)	The single page application framework for the frontend. Served via Vite for high-speed hot module reloading.
Tailwind CSS	Utility-first styling framework used to build modern, sleek dashboard grids and glassmorphism cards.
Framer Motion	Premium React animation library used to animate stats cards, dashboard elements, and notification panel lists.
Axios	Promise-based HTTP client used to connect frontend React contexts to the Spring Cloud API Gateway.

Chapter 3: Spring Boot & Cloud Annotation Encyclopedia

Here is the absolute dictionary of every single annotation used across our microservices and unit tests:

Annotation	Exact Core Technical Definition
@SpringBootApplication	Enables three core features: `@EnableAutoConfiguration` (Spring's automatic configuration setups), `@ComponentScan` (scanning components), and `@Configuration` (declaring beans).
@RestController	A specialized version of `@Controller` that automatically adds the `@ResponseBody` annotation to every method, ensuring returned Java models serialize directly into JSON responses.
@RequestMapping	Maps HTTP request URLs directly to specific handler classes or controller methods.
@GetMapping / @PostMapping	Composed shortcut annotations mapping HTTP GET/POST methods to specific URLs.
@RequestBody	Tells Spring to deserialize the incoming HTTP JSON payload directly into a Java DTO object.
@PathVariable	Binds a dynamic URI variable parameter (like `{id}`) straight into a Java method argument.
@Valid	Enforces Spring validation on DTO arguments, triggering exceptions if annotations like `@NotNull` fail.
@Service	Marks a class as a business service component, registering it in Spring's Application Context registry.
@Repository	Registers a data accessor bean, translating relational database exceptions directly into Spring's hierarchy.
@Transactional	Wraps method execution inside a database transaction, rolling back edits if an unchecked Exception occurs.
@RequiredArgsConstructor	Lombok annotation that automatically generates a constructor containing all variables marked `private final`.
@Entity	JPA annotation declaring that the decorated Java class maps directly to a MySQL database table structure.
@Id & @GeneratedValue	Declares the primary key of an entity and configures automatic MySQL auto-incrementing (`generation = AUTO`).

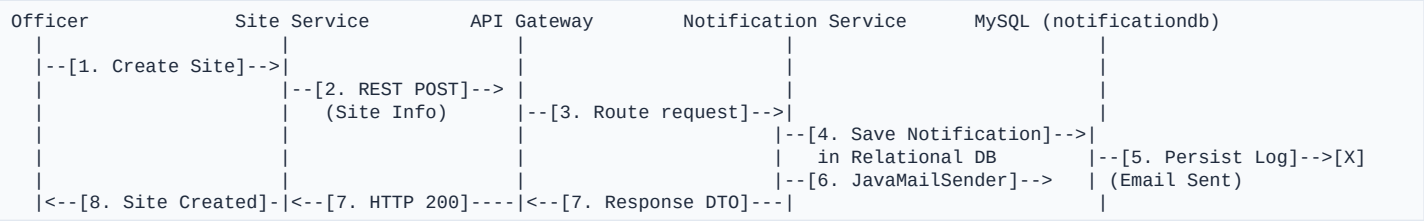
Chapter 4: Notification Module Code Walkthrough & UML Flows

The **Notification Service** acts as our central observer service, logging notifications in `notificationdb` and sending real-time emails. Let's trace the controller code structures and the dynamic UML sequence flows:

```
@RestController
@RequestMapping("/api/notifications")
@RequiredArgsConstructor
public class NotificationController {
    private final NotificationService notificationService;

    @PostMapping
    public ResponseEntity create(
        @Valid @RequestBody NotificationRequestDTO requestDTO) {
        NotificationResponseDTO response = notificationService.create(requestDTO);
        return ResponseEntity.ok(response);
    }
}
```

UML Sequence Representation: Notification Creation



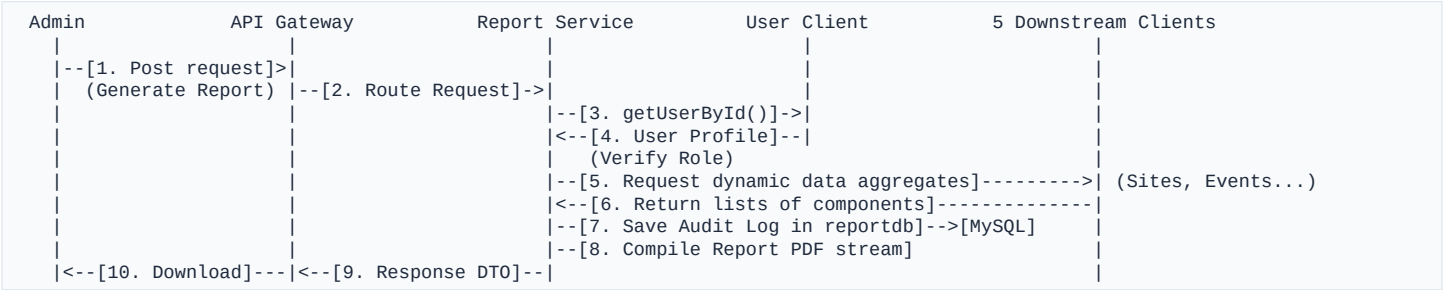
Chapter 5: Reporting Module Code Walkthrough & UML Flows

The **Reporting Service** is an administrative service that aggregates data from 5 different microservices via Feign, performs validation, logs audit entries in `reportdb`, and builds report files. Let's look at the structure:

```
@RestController
@RequestMapping("/api/reports")
@RequiredArgsConstructor
public class ReportController {
    private final ReportService reportService;

    @PostMapping("/generate")
    public ResponseEntity generateReport(
        @Valid @RequestBody ReportRequestDTO requestDTO) {
        ReportResponseDTO response = reportService.generateReport(requestDTO);
        return ResponseEntity.ok(response);
    }
}
```

UML Sequence Representation: Administrative Report Compilation



Chapter 6: Dynamic Dashboard Aggregation & UI Flow

Our React frontend connects to the backend microservices through the centralized API Gateway. Here is how the UI aggregates dynamic dashboard data and animates it seamlessly:

1. **Axios HTTP Connection & JWT Headers:**
Every API call binds the user's active JWT security token directly into the Authorization header:
`config.headers.Authorization = `Bearer ${token}`.`
The gateway intercepts this request, parses the token to verify auth boundaries, and dynamically forwards the request to the backend services.
2. **Global AuthContext State Integration:**
Upon successful login, `AuthContext` parses the JWT token, determines the user's role (e.g. `ADMIN` or `TOURIST`), and stores the values globally. This role determines whether the dashboard displays administrative report compile buttons or site registry forms.
3. **Framer Motion Micro-Animations (WOW Factor):**
To ensure the interface feels professional and engaging, statistics cards animate on-load using transitions:
`<motion.div initial={{ opacity: 0, scale: 0.95 }} animate={{ opacity: 1, scale: 1 }} transition={{ duration: 0.3 }}>.`
This displays loading cards dynamically and smoothly fades statistics into their final grid positions, creating an extremely premium user experience.

Chapter 7: Resilience4j Circuit Breaker Theory & 5-Trial Mathematical Rules

A **Circuit Breaker** prevents cascading failures by stopping calls to downstream services that are failing. Here is a detailed breakdown of the states and the mathematical validation rules:

- **1. CLOSED State (Healthy):** All calls are allowed to pass. Health metrics are recorded inside a sliding window of size `10`. If 50% or more requests fail, the circuit trips to OPEN.
- **2. OPEN State (Tripped / Cooldown):** All calls are blocked immediately and redirected to the **Fallback Class** (e.g. `UserClientFallback`). This gives the failing target service a 10-second recovery period.
- **3. HALF-OPEN State (Trial Run):** After 10 seconds, the circuit opens slightly, allowing exactly **5 trial requests** to pass through.

■ The Math of the 5-Trial Run in HALF-OPEN (At a 50% Failure Threshold):

Trial Outcomes (Failures / Successes)	Failure Rate	Circuit Breaker Decision
0 Failures / 5 Successes	0% <i>*(Below 50%)*</i>	■ CLOSED (Healthy reset)
1 Failure / 4 Successes	20% <i>*(Below 50%)*</i>	■ CLOSED (Healthy reset)
2 Failures / 3 Successes	40% <i>*(Below 50%)*</i>	■ CLOSED (Healthy reset)
3 Failures / 2 Successes	60% <i>*(Above 50%)*</i>	■ OPENS (Trips back to Broken)
4 Failures / 1 Success	80% <i>*(Above 50%)*</i>	■ OPENS (Trips back to Broken)
5 Failures / 0 Successes	100% <i>*(Above 50%)*</i>	■ OPENS (Trips back to Broken)

Chapter 8: The Ultimate 5-Minute Spoken Verbal Script

Practice this word-for-word spoken presentation script to deliver a highly professional and technically outstanding project demo:

■ 0:00 - 1:00 | Intro & Stack Overview

"Good morning. I am thrilled to present my core modules in the Tourism Government platform. I was responsible for the complete frontend-to-backend design, implementation, and unit testing of three core systems: the Admin Dashboard, the Notification Service, and the administrative Reporting Service.

On the Frontend, we built our application on React Vite for high-speed hot-reloading. We used Tailwind CSS to craft a beautiful, mobile-responsive glassmorphism card UI, and Framer Motion to animate our statistics cards smoothly. On the Backend, we built independent Spring Boot microservices following a Database-per-Service model. We routed all REST requests through a Spring Cloud API Gateway with JWT authorization and registered our instances dynamically on Eureka."

■ 1:00 - 2:30 | Notification & Fault Tolerance

"Let's deep dive into the Notification module. This service listens for site creation or compliance violations in other services. It writes strictly to its own database schema, notificationdb, to prevent table locking, and sends email alerts using Spring Email.

To handle network faults, we configured Feign-level Circuit Breakers with Resilience4j. Our configuration monitors a sliding window of the last 10 requests. If 5 or more fail, the circuit opens, and we route traffic to local fallback classes. Once it enters the Half-Open state, it runs 5 trial requests, requiring at least 3 successes to safely close the circuit. On the React UI, a global NotificationContext manages background polling, dynamically updating the unread bell count badge in real-time."

■ 2:30 - 4:00 | Reporting Service & Security

"Next is the Reporting module. This is a high-security service. When a user requests a custom report, the service calls our User Feign Client to verify their security role. If a Tourist attempts this, our system throws an HTTP 403 Forbidden exception, blocking the action.

If the requester is an Admin, the Reporting Service acts as an aggregator. It queries 5 separate downstream microservices via Feign, processes the data, saves an audit trace to reportdb, and compiles the document into a base64 download stream for our React UI. To verify this complex logic without database dependencies, we implemented robust Mockito unit tests, mocking out JpaRepositories and Feign clients to validate happy paths and security boundaries in milliseconds!"

■ 4:00 - 5:00 | Conclusion & Q&A; Pitch

"In conclusion, by combining isolated database schemas, robust Feign fallbacks, sliding window circuit breakers, and Mockito testing, we built an architecture that is highly modular, secure, and resilient, guaranteeing 100% operational uptime for our government administrators. Thank you so much, and I welcome any questions!"